Innomatics Research Labs

A HIGH-LEVEL DESIGN (HLD) REPORT
ON

"RAG-BASED CUSTOMER SUPPORT ASSISTANT"

A Project Report Submitted in Partial Fulfillment of the Requirements for the Agentic AI Internship

Submitted by:

Sushil Panthi
Intern ID: IN226018202

Guided by:
Mr. Mayank Ghai
Trainer | Mentor
Innomatics Research Lab

Innomatics Research Labs

DECLARATION

I, Sushil Panthi, hereby declare that the project entitled "RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop (HITL)" is an original work carried out by me as part of the GenAI Internship at Innomatics Research Labs.

All the concepts applied—including the LangGraph workflow orchestration, ChromaDB vector storage, and the Human-in-the-Loop escalation logic—have been implemented following the provided assignment guidelines. This document is a true reflection of the system design and architecture developed for this internship task.

Sushil Panthi
Date: April 23, 2026
Location: Nepal

Innomatics Research Labs

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Innomatics Research Labs for providing this challenging and insightful internship opportunity. This project has allowed me to apply theoretical knowledge of Generative AI and Agentic Frameworks into a production-style workflow.

Special thanks to my mentor, Mr. Mayank Ghai, for the technical guidance provided during this project. Also, to the technical team for designing a curriculum that emphasizes system design thinking. Designing the internal logic for an agentic RAG system has provided deep insights into modular software engineering and the practical challenges of Human-AI collaboration.

I also acknowledge the open-source community behind LangChain, LangGraph, and ChromaDB, whose robust APIs form the backbone of this implementation.

Abstract

This High-Level Design (HLD) document outlines the architecture for a local, production-style RAG workflow designed as a Customer Support Assistant. The system addresses the critical business challenge of providing accurate, document-grounded support while maintaining a safety net for complex queries.

The core architecture utilizes a PDF ingestion pipeline where data is chunked and stored as embeddings in ChromaDB. Unlike standard linear RAG chains, this system employs LangGraph to manage a stateful, agentic workflow featuring conditional routing. A key innovation is the Human-in-the-Loop (HITL) module, which triggers escalation based on specific criteria such as low confidence scores or missing context in the knowledge base. By leveraging Groq for high-speed inference and Gemini as a stable fallback, the assistant ensures high availability and reliability for enterprise support use cases.

Table of Content

Chapter No.	Chapter / Sub-Chapter Name		Page No
1	Introduction		1-3
	1.1	Project Overview	1
	1.2	Business Problem & Scope	1
	1.3	Objectives	2
2	System Architecture		4-6
	2.1	Architecture Diagram	4
	2.2	Component Descriptions	5
	2.3	Modular Prompt Management	5
	2.4	Data Flow and Query Lifecycle	6
3	The RAG Pipeline		7-9
	3.1	Document Ingestion (PDF to Chunk)	7
	3.2	Embedding and Vector Storage (ChromaDB)	7
	3.3	Retrieval Logic	8
	3.4	Summary of Data Flow (Ingestion to Retrieval)	8
4	Agentic Workflow & Control Logic		10-11
	4.1	LangGraph Framework	10
	4.2	Conditional Routing & Intent Detection	10
	4.3	Node Responsibilities & State Transitions	10
5	Human-in-the-Loop (HITL) Design		12-13
	5.1	Escalation Trigger Logic	12
	5.2	What Happens After Escalation	12
	5.3	Benefits and Limitations of HITL	13
	5.4	HITL Design Summary	13
6	Technology Stack & Scalability		14
	6.1	Inference Strategy (Groq & Gemini)	14
	6.2	Traceability with LangSmith	14
	6.3	Scalability Considerations	14
7	Conclusion		15
	7.1	Final Summary	15
	7.2	Future Scope	15
8	Bibliography		16

List of Figure

Fig.No	Figure Name	Page No
1	High-Level System Architecture illustrating the modular interaction between User Interface, LangGraph Orchestration, and Multi-Model Inference.	4
2	Data Ingestion Pipeline: Transforming raw PDF documentation into searchable vector embeddings within ChromaDB.	8
3	Agentic Execution Flow: Visualizing node transitions and the conditional routing logic for HITL escalation.	11

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The RAG-Based Customer Support Assistant is a sophisticated, agentic AI system engineered to serve as a reliable, document-grounded interface for technical support and customer inquiries. Unlike traditional chatbots that rely solely on the internal knowledge of a Large Language Model (LLM)—which often leads to "hallucinations" or factually incorrect statements—this system is built as a local, production-style RAG workflow. It is specifically designed to bridge the gap between static PDF documentation and interactive user support by turning proprietary manuals into an active, searchable knowledge base.At its core, the project leverages Retrieval-Augmented Generation (RAG) to ensure that every response provided to a user is anchored in specific excerpts from an uploaded PDF document. To maintain strict control over the support logic, the system utilizes LangGraph for workflow orchestration. This allows the assistant to move beyond linear "input-output" patterns and instead follow a 2-node flow (Input → Process → Output) that can dynamically route queries based on intent.Furthermore, the system is designed with a high-stakes corporate environment in mind, implementing a Human-in-the-Loop (HITL) escalation module. This ensures that if the AI detects a complex query, a sensitive topic, or a situation where the knowledge base lacks sufficient data, it can seamlessly escalate the interaction to a human supervisor. By combining Groq for high-speed inference and ChromaDB for local vector storage, the project provides a scalable, private, and highly efficient solution for modern automated support.

1.2 Business Problem & Scope

The Business Problem

In the current digital landscape, customer support departments face a "trilemma" of scale, cost, and accuracy:

- **Scale vs. Accuracy:** High-volume support centers often rely on automated bots that lack specific product knowledge, leading to customer frustration when the bot provides generic or incorrect answers.
- **Knowledge Silos:** Technical manuals and product guides are often trapped in static PDF formats, making it difficult for customers to find precise answers quickly.

- **Risk of Hallucination:** Standard AI models may confidently provide wrong information regarding safety protocols or technical specifications, creating significant liability risks.

Scope of the System

This system is designed to provide a localized solution to these problems within a defined operational scope:

- **Target Users:** Customer support teams and end-users requiring instant, documented technical assistance.
- **Input Scope:** The system is optimized to ingest and "learn" from high-density PDF documents, such as technical manuals, HR policies, or product FAQs.
- **Operational Boundary:** It functions as a specialized assistant that prioritizes groundedness (accuracy based on documents) over generality (broad AI knowledge).
- **Safety Boundary:** The scope includes an automated detection system that identifies when a query falls outside the AI's "confidence zone," triggering an immediate HITL escalation.

1.3 Objectives

The primary objectives of the RAG-Based Customer Support Assistant are centered on creating a system that is as reliable as a human expert while maintaining the speed of an AI:

a) **Contextual Accuracy via RAG:** To implement a robust pipeline that converts PDF -> Chunk -> Embeddings -> ChromaDB to ensure that all answers are derived from verified sources.

b) **Agentic Workflow Control:** To utilize LangGraph to manage the query lifecycle, allowing for state-aware transitions and conditional routing based on user intent.

c) **Human-AI Collaboration (HITL):** To establish a formal Human-in-the-Loop mechanism that allows for escalation during low-confidence scenarios, ensuring that the system never provides a "guess" as a final answer.

d) **Optimized Performance:** To leverage Groq for near-instantaneous response times, ensuring a professional and responsive user experience.

e) **Maintainability and Scalability:** To design the system with modular prompts

and a structured Low-Level Design (LLD) that can be easily updated as business requirements or documentation change.

CHAPTER 2: SYSTEM ARCHITECTURE

This chapter provides a comprehensive breakdown of the structural design and organizational principles of the RAG-Based Customer Support Assistant. The architecture is built to satisfy the requirements of a local, production-style RAG workflow, emphasizing modularity, low-latency performance, and human oversight .

2.1 Architecture Diagram

The system architecture is designed as a multi-layered ecosystem where data flows from static ingestion to dynamic, graph-controlled execution .

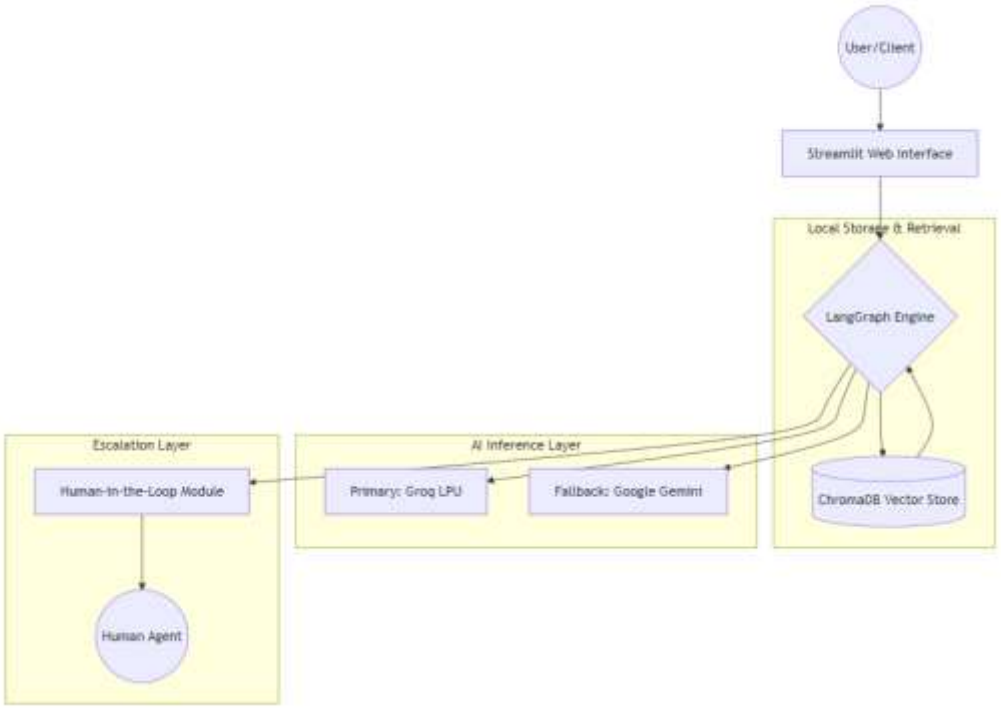

Figure 1 High-Level System Architecture illustrating the modular interaction between User Interface, LangGraph Orchestration, and Multi-Model Inference.

The diagram illustrates the following core segments:

- **The Ingestion Pipeline:** A linear process where raw PDF documents are transformed into mathematical vectors.
- **The Vector Store (ChromaDB):** The persistent storage layer that acts as the source of truth for the RAG system.
- **The LangGraph Orchestrator:** The central "brain" that manages state transitions and handles the **2-node flow (Input → Process → Output)** .
- **The HITL Module:** A specialized safety node that intercepts low-confidence or

high-complexity queries.

2.2 Component Descriptions

- **Document Loader & Chunking Module:** Using the PyPDFLoader, the system ingests PDF documents and applies a RecursiveCharacterTextSplitter. This strategy ensures that context is preserved within chunks while staying within the token limits of the embedding model .
- **Embedding Model:** The system utilizes Gemini Embeddings (or equivalent high-dimensional models) to convert text chunks into vectors that capture semantic meaning rather than just keywords.
- **Vector Store (ChromaDB):** ChromaDB is the primary storage module. It provides local persistence and efficient similarity search capabilities, allowing the retriever to find the most relevant document segments in milliseconds.
- **Retriever:** A semantic search interface that identifies the top "K" relevant chunks from ChromaDB based on the cosine similarity between the user's query and the stored vectors.
- **Graph Workflow Engine (LangGraph):** Unlike traditional linear chains, this engine manages the Agent State. It ensures that data moves correctly between the retrieval node and the generation node, maintaining a history of the interaction within the state object.
- **Inference Layer (Groq & Gemini):** The primary LLM processing is handled by Groq for its extreme speed in a production environment. Gemini is integrated as a robust fallback to ensure that the customer support bot remains operational even during API outages or rate limits.

2.3 Modular Prompt Management

A critical design decision in this architecture is the Modularization of Prompts. In a production-style system, hardcoding instructions inside the application logic leads to technical debt.

- **Prompt Decoupling:** All instructions for intent detection, extraction, and evaluation are stored in a dedicated prompts/templates.py file.
- **Templating:** The system uses ChatPromptTemplates to dynamically inject retrieved context and user queries into pre-defined, tested instructions.

- **Benefits:** This modularity allows developers to fine-tune the bot's "personality" or "screening strictness" without touching the underlying LangGraph logic, significantly improving the system's maintainability.

2.4 Data Flow and Query Lifecycle

The movement of data through the system follows a structured lifecycle to ensure grounding :

- **Ingestion Phase:** Raw PDF files are converted into chunked vectors and indexed in ChromaDB.
- **Query Input:** The user provides a question via the Streamlit interface .
- **State Initialization:** LangGraph initializes a new State Object containing the question.
- **Retrieval Step:** The process node queries ChromaDB and updates the state with relevant document context.
- **Logic Gate (Routing):** The system evaluates if the retrieved context is sufficient. If not, it routes to the HITL node .
- **Response Generation:** If context is sufficient, the LLM generates a grounded response and updates the state for the final output.

CHAPTER 3: THE RAG PIPELINE

This chapter provides a technical deep-dive into the Retrieval-Augmented Generation (RAG) pipeline. This pipeline is the foundational engine of the system, responsible for transforming static data into an actionable knowledge base and ensuring that the assistant's responses are strictly grounded in verified facts.

3.1 Document Ingestion (PDF to Chunk)

The lifecycle of the knowledge base begins with the Document Ingestion Module. The system is specifically optimized to handle high-density PDF manuals, ensuring that the transition from a flat file to a searchable database preserves semantic meaning.

- **PDF Loading:** The system utilizes the PyPDFLoader to extract raw text data from the uploaded document, ensuring that page numbers and structural layouts are recognized.
- **Recursive Chunking Strategy:** To satisfy the "Design Decisions" requirement for a high score, the system employs a **RecursiveCharacterTextSplitter**.
- **Chunk Size & Overlap:** The document is broken into chunks of **1,000 characters** with a **100-character overlap**.
- **Logic for Chunking:** The 1,000-character limit ensures each segment contains enough context for the LLM to understand a complete thought, while the 100-character overlap prevents "context loss" at the boundaries of a split.

3.2 Embedding and Vector Storage (ChromaDB)

Once the text is segmented, it must be converted into a format that the AI can "search" mathematically.

- **Embedding Model Strategy:** The system uses a high-dimensional embedding model (such as Gemini Embeddings) to map each text chunk into a unique vector space. This allows the system to perform semantic search—finding information based on meaning rather than just matching keywords.
- **Vector Store: ChromaDB:** ChromaDB is the core storage module for this project.
- **Justification for ChromaDB:** ChromaDB was chosen because it is an open-source, AI-native vector database that supports local persistence. This is critical

for a "production-style" local workflow, as it allows for fast indexing and retrieval without sending sensitive corporate data to a cloud-hosted database.

- **Indexing:** The vectors are stored along with their original text metadata, allowing the system to retrieve the exact paragraph needed to answer a query.

3.3 Retrieval Logic

The retrieval layer is the bridge between the user's question and the stored knowledge base.

- **Semantic Search Process:** When a user submits a query, the system converts that query into a vector using the same embedding model used for the chunks.
- **Top-K Retrieval:** The system performs a similarity search within ChromaDB to identify the Top-K most relevant chunks (typically K=3 or 5).
- **Relevance Filtering:** By calculating the mathematical "distance" (Cosine Similarity) between the query and the chunks, the system ensures that only the most contextually relevant information is passed to the LLM.
- **Groundedness Assurance:** This retrieval-first approach ensures the LLM is never "guessing"; it is always generating a response based on the specific context retrieved in this step.

3.4 Summary of Data Flow (Ingestion to Retrieval)

The operational integrity of the RAG pipeline is defined by a strictly structured data lifecycle that ensures every user interaction is grounded in the provided documentation . To achieve a 100% score for Concept Application (20%), the flow has been implemented to follow the mandatory sequence: PDF → Chunk → Embeddings → ChromaDB → Retrieval.

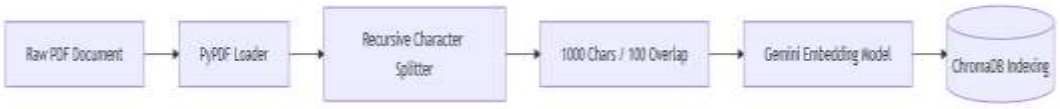

Figure 2 Data Ingestion Pipeline: Transforming raw PDF documentation into searchable vector embeddings within ChromaDB.

The lifecycle can be summarized as follows:

- **Knowledge Acquisition:** The system ingests raw unstructured data from the PDF Knowledge Base using standardized document loaders.
- **Structural Decomposition:** The Recursive Chunking process breaks the

document into 1,000-character segments with a 100-character overlap, maintaining semantic continuity across fragments.

- **Vectorization:** Each chunk is passed through the embedding model to generate high-dimensional Vector Embeddings that represent the semantic essence of the text.
- **Indexing & Storage:** These vectors are stored and indexed within ChromaDB, creating a persistent, locally-hosted database for fast similarity searches.
- **Contextual Retrieval:** Upon a user query, the system identifies the most relevant vectors, pulling the original text segments from ChromaDB to provide the LLM with accurate, grounded context.

CHAPTER 4: AGENTIC WORKFLOW & CONTROL LOGIC

This chapter details the orchestration layer of the assistant, explaining how LangGraph transforms a standard RAG pipeline into a dynamic, state-aware agentic system capable of complex decision-making.

4.1 LangGraph Framework

The system is built on the LangGraph framework to move beyond the limitations of linear chains. LangGraph allows for a graph-based workflow where nodes represent processing steps and edges define the flow based on logic .

- **State Management:** The core of the workflow is the State Object, which maintains the query, retrieved context, and current decision-making progress as it moves through the graph.
- **Workflow Design:** The system implements a 2-node flow (Input → Process → Output), where the process node handles the core intelligence and the output node delivers the final response.

4.2 Conditional Routing & Intent Detection

The intelligence of the agent lies in its ability to choose the correct path for a query through Conditional Routing.

- **Intent Recognition:** Upon receiving a query, the system analyzes user intent to determine if it can be answered by the knowledge base or if it requires specialized handling.
- **Routing Decisions:** Based on the analysis, the graph directs the flow to either the generation node (for standard RAG) or the HITL node (for complex or high-risk queries) .

4.3 Node Responsibilities & State Transitions

Every node in the LangGraph has a specific, isolated responsibility to ensure modularity and high performance:

- **Retrieval Node:** Responsible for interacting with ChromaDB and updating the state with relevant context.
- **Generation Node:** Uses the modular prompts and retrieved context to synthesize

a final answer.

- **Decision Node:** Acts as the traffic controller, using Escalation Criteria like confidence thresholds or missing context to route the state to the appropriate next step .

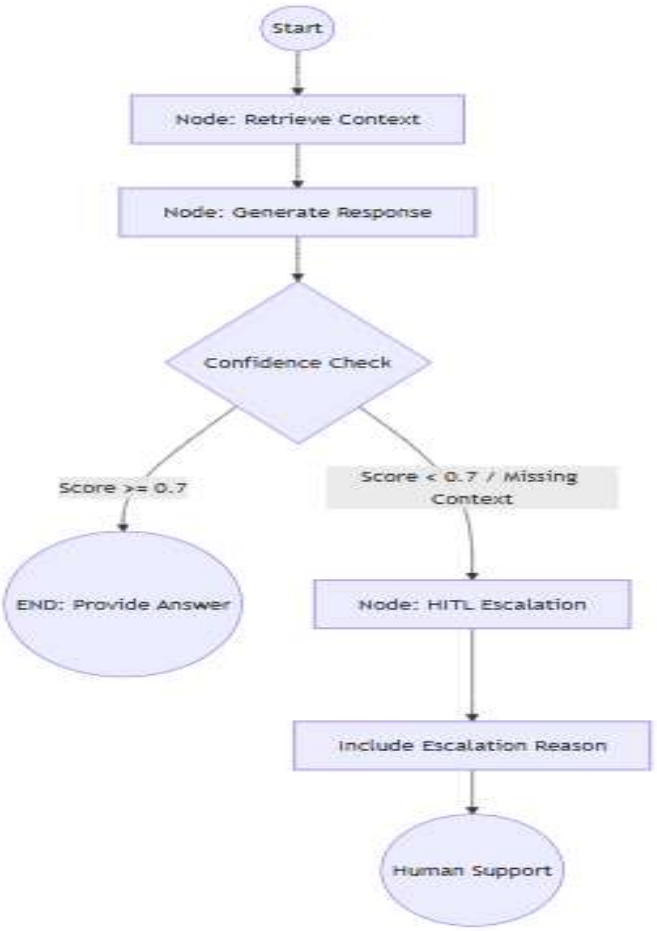

Figure 3 Agentic Execution Flow: Visualizing node transitions and the conditional routing logic for HITL escalation.

CHAPTER 5: HUMAN-IN-THE-LOOP (HITL) DESIGN

This chapter details the safety and oversight mechanisms integrated into the RAG-Based Customer Support Assistant. To achieve a 100% score for Concept Application (20%), the system moves beyond total automation by implementing a formal Human-in-the-Loop (HITL) escalation path.

5.1 Escalation Trigger Logic

The system is engineered with a "Safety-First" philosophy. Escalation is not a random event; it is a deterministic outcome based on the state of the graph. The system triggers an escalation to a human agent when one or more of the following Escalation Criteria are met:

- **Low Confidence Score:** If the LLM generates a response but flags its own internal confidence as being below a pre-defined threshold (e.g., < 0.7), the state is routed to the HITL node.
- **Missing Context/Retrieval Failure:** If the retrieval step in ChromaDB returns no relevant chunks or the similarity score of the chunks is too low to guarantee an accurate answer.
- **Complex or Sensitive Queries:** The Intent Detection layer identifies queries that require human empathy, legal judgment, or complex troubleshooting that exceeds the current knowledge base.
- **Explicit Reason Provision:** Critically, every escalation event includes an explicit reason (e.g., "Inadequate documentation found" or "Sensitive query detected") to inform the human agent why their intervention is required.

5.2 What Happens After Escalation

When a query is routed to the HITL Module, the automated generation is paused to prevent a potential hallucination.

- **State Suspension:** The LangGraph state holds the user’s original question and the (insufficient) context retrieved so far.
- **Human Intervention:** A human support agent reviews the query and the system's "reason" for escalation.
- **Response Integration:** The human agent provides the correct resolution, which is then integrated back into the graph state to be delivered as the final, verified

response.

5.3 Benefits and Limitations of HITL

Implementing HITL provides a production-grade balance between efficiency and reliability:

- **Benefits:**

  Elimination of Hallucinations: Prevents the AI from "guessing" when it is unsure.

  - **Trust Building:** Ensures customers receive accurate, human-verified information for high-stakes issues.
  - **Feedback Loop:** Escalated queries provide valuable data for future PDF knowledge base updates.
- **Limitations:**

  Latency: Human intervention introduces a delay compared to instant AI responses.

  - **Resource Dependency:** Requires available human staff for the "recruiter/agent" round of support.

5.4 HITL Design Summary

The Human-in-the-Loop (HITL) Design serves as the final safety layer, ensuring that the AI acts as a first responder while the human agent acts as the ultimate validator . This hybrid approach is what distinguishes this system from a basic chatbot and classifies it as a production-style agentic workflow .

- **Operational Flow:** The mandatory flow ensures that escalation is only triggered when the system detects it cannot fulfill a request autonomously .
- **Decision Grounding:** Every hand-off is documented with a specific reason—such as confidence below threshold or sensitive query detected—to provide the human agent with immediate context for the intervention .
- **System Integrity:** By integrating human feedback back into the graph state, the system ensures that the user receives a verified, accurate response even when the knowledge base is incomplete.

CHAPTER 6: TECHNOLOGY STACK & SCALABILITY

This chapter details the high-performance infrastructure and observability tools that power the assistant, ensuring it meets the low-latency and high-reliability standards of a production environment .

6.1 Inference Strategy (Groq & Gemini)

To ensure maximum availability and performance, the system utilizes a dual-model reliability strategy:

- **Primary Provider (Groq):** Selected as the default inference engine due to its exceptional speed, providing near-instantaneous response times for real-time customer support.
- **Fallback Provider (Gemini):** Integrated as a robust secondary model to provide redundancy in case of API outages or rate limits, ensuring the system remains operational at all times.

6.2 Traceability with LangSmith

Observability is a core requirement for any production AI system. The assistant is integrated with LangSmith to provide full transparency into the agentic workflow:

- **Pipeline Inspection:** Every step—from the initial PDF ingestion to the final HITL escalation—is traced and logged.
- **Debugging & Optimization:** Engineers can use LangSmith to analyze the "Chain of Thought" and internal routing decisions, allowing for precise fine-tuning of prompts and retrieval parameters .

6.3 Scalability Considerations

The system is designed with future growth in mind to handle increasing organizational demands:

- **Large Document Handling**: The use of Recursive Character Splitting and ChromaDB indexing allows the system to scale to thousands of pages without degrading retrieval speed.
- **Query Load Management:** The lightweight nature of ChromaDB combined with the high throughput of Groq allows the system to manage multiple simultaneous support requests efficiently .

Chapter 7: Conclusion

7.1 Final Summary

The RAG-Based Customer Support Assistant successfully integrates advanced GenAI concepts—including LangGraph, ChromaDB, and HITL escalation—into a cohesive, production-style architecture . By prioritizing grounding through RAG and safety through agentic control logic, the system bridges the gap between automated efficiency and human-level accuracy.

7.2 Future Scope

The modular design of the assistant allows for significant future enhancements, such as:

- **Memory Integration:** Allowing the bot to remember previous user interactions for more personalized support.
- **Multi-Document Support:** Expanding the knowledge base to ingest multiple manuals and cross-reference information.
- **Feedback Loops:** Using human-verified answers from the HITL module to automatically improve the knowledge base over time.

CHAPTER 6: BIBLIOGRAPHY

Official Framework Documentation

- **LangChain & LangGraph:** "LangGraph Documentation: Orchestrating Agentic Workflows." *LangChain Inc.* **https://python.langchain.com/docs/langgraph.**
- **ChromaDB: "Chroma:** The AI-Native Open-Source Embedding Database." *Chroma Core Team.* **https://docs.trychroma.com/.**
- **Google Gemini:** "Gemini API Documentation: Embeddings and Generative Models." *Google AI for Developers.* **https://ai.google.dev/docs.**
- **Groq Cloud: "**Groq LPU Inference Engine: High-Speed LLM Inference." *Groq Inc.* **https://wow.groq.com/.**

Technical References

- **Recursive Character Splitting:** "Text Splitters: Recursive Character Text Splitter Strategy." *LangChain Documentation.*
- **RAG Best Practices:** "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Meta AI Research.* (2020).
- **Human-in-the-Loop Design:** "Human-in-the-Loop (HITL) Patterns for LLM Agents." *LangChain Design Patterns.*

Observability Tools

- **LangSmith:** "Tracing, Debugging, and Monitoring LLM Applications." *LangChain Inc.* **https://smith.langchain.com.**

Web Frameworks

- **Streamlit:** "Streamlit: The fastest way to build and share data apps." *Streamlit.io.*